KRR Lab 4 - Smart Home Rules in Drools

From SWRL to a production-rule engine

Lecture 3, slides 25–34: the smart-home ontology and its SWRL rules

Where we are

In Lecture 3 we wrote smart-home automation rules in **SWRL** over a small OWL ontology:
rooms, motion sensors, lux sensors, lights.

```
MotionSensor(?s) ^ hasState(?s, "on") ^ locatedIn(?s, ?r)  ->  OccupiedRoom(?r)
```

Slide 33 listed what goes wrong, because SWRL is **monotonic**:

- a conclusion is **never withdrawn**: motion stops, but the room stays "occupied"
- **no negation as failure**: we cannot say "no motion sensor in the room is on"
- conflicting conclusions (`OccupiedRoom` and `EmptyRoom`) make the whole ontology **inconsistent**
- no way to say **who wins** when `DesiredOn` and `SuppressedAction` both hold

Today

Run the **same rules** in **Drools**, a *production-rule* engine, and see how it handles each of these problems.

What is Drools?

Drools is an open-source **production-rule system** (business rules engine) for Java,
part of the Apache KIE project.

Main ingredients

- **Working memory (WM)**: the current set of **facts**, i.e. plain Java objects (`Light` , `Room` , ...)
- **Rules**: `when <conditions> then <actions>` , written in **DRL** (Drools Rule Language)
- **Rule engine (Phreak)**: matches rules against the facts and builds the **agenda** of rules ready to fire
- **Agenda**: decides **which** rule fires **next** (conflict resolution, priorities)

Production rules = condition → action

- The `then` part is **Java code**: it can insert, change or delete facts, or call external services
- So a rule can *derive* knowledge (like SWRL) **and** *act* (like a Home Assistant automation)

Drools vs. SWRL

	SWRL (+ OWL reasoner)	Drools
Kind of system	logic: rules extend an ontology	production rules: condition → action
Semantics	**monotonic**: facts only accumulate	**non-monotonic**: facts can be changed and retracted
World assumption	**open world**	**closed world**: what is not in WM is false
Negation	no negation as failure	`not`, `exists`, `forall` over working memory
Withdrawing a conclusion	never	`insertLogical`: retracted automatically (truth maintenance)
Contradiction	ontology becomes **inconsistent**	just more facts: a rule can detect and handle it
Rule head	only atoms (class / property assertions)	arbitrary Java: modify facts, call services, print
Order of rule application	irrelevant (same result)	matters: **salience**, **agenda groups**
Data model	OWL classes and properties	Java classes and fields

Trade-off: SWRL gives a declarative meaning, independent of order. Drools gives control, but *you* are responsible for the order in which rules act.

Anatomy of a DRL rule

```
rule "R3 Occupied and dark => desire light on"       // name
    salience 0                                       // attributes (optional)
when                                                 // LHS: patterns over facts
    OccupiedRoom( $r : room )                        //   bind $r
    DarkRoom( room == $r )                           //   join on $r
    Light( locatedIn == $r, $l : id )                //   every light in that room
then                                                 // RHS: Java code
    insertLogical( new DesiredOn( $l ) );
end
```

SWRL	DRL
`MotionSensor(?s)`	`MotionSensor( ... )`, a **pattern**
`hasState(?s, "on")`	`MotionSensor( hasState == "on" )`, a **field constraint**
`locatedIn(?s, ?r)`, first occurrence of `?r`	`$r : locatedIn` (binds `$r`)
`locatedIn(?l, ?r)`, `?r` already bound	`Light( locatedIn == $r )` (join)
`swrlb:lessThan(?lx, 20)`	`LuxSensor( measuredLux < 20 )`
`-> OccupiedRoom(?r)`	`insertLogical( new OccupiedRoom( $r ) );`
(not expressible)	`not SuppressedAction( device == $l )`

Changing the working memory

From Java	In a rule's RHS	Effect
`session.insert(f)`	`insert( f )`	add a fact, stays until deleted
—	`insertLogical( f )`	add a fact **justified** by this rule's match
`session.update(handle, f)`	`modify( f ) { setX(...) }`	tell the engine a fact changed
`session.delete(handle)`	`delete( f )`	remove a fact

`insertLogical` and truth maintenance (TMS)

- The fact stays in WM only while **at least one** rule match that inserted it still holds.
- When the conditions stop holding, the TMS **retracts** the fact automatically, and that can cascade to facts derived from it.
- This is exactly what SWRL lacks: *motion stops* → `OccupiedRoom` *disappears* → `DesiredOn` *disappears*.

Looking at working memory

`session.getObjects(filter)` returns the facts currently in WM.
`SmartHomeApp.printState()` uses it to print the derived facts after each step.

The rule execution cycle

```
Java: insert / update / delete facts
                  |
                  v
+--> 1. MATCH     propagate the change through the rule network (Phreak)
|                 -> complete matches become ACTIVATIONS on the agenda
|                 v
|    2. SELECT    conflict resolution: pick ONE activation
|                 (focused agenda group first, then highest salience)
|                 v
|    3. FIRE      execute its RHS (then-part)
|                 v
+--- 4. the RHS inserted / modified / deleted facts?  -> back to 1
                  |
                  v   agenda empty (fixpoint)
fireAllRules() returns the number of rules fired
```

- **Refraction**: a rule fires **once per match**. It fires again only if one of the matched facts is modified.
- Firing a rule can **create** new activations and **cancel** existing ones.
- Phreak is **lazy**: a rule's matches are computed when the agenda needs them, not at insertion time.

Rule priorities: why the order matters

The smart-home rules come in two layers:

- **Inference** (R1–R6, C1): derive `OccupiedRoom`, `DarkRoom`, `DesiredOn`, `SuppressedAction`, `Inconsistency`, …
- **Actuation** (A1–A4): *act* on these facts, e.g. turn a light on.

```
rule "A1 Actuate: turn light on"
when
    DesiredOn( $l : device )
    not SuppressedAction( device == $l )        // negation as failure
    $light : Light( id == $l, hasState == "off", $r : locatedIn )
    not Inconsistency( room == $r )
then
    modify( $light ) { setHasState( "on" ) };
end
```

Danger: if A1 fires *before* R4 has derived `SuppressedAction(lamp2)`, the `not` is still true,
and the bedroom light goes on during quiet hours. Once it has fired, nothing switches it back off.

→ Actuation must wait until inference has reached a **fixpoint** (stratification).
Drools offers two mechanisms: **salience** and **agenda groups**.

Mechanism 1: salience

`salience` is an integer priority, default `0`. **Among the activations on the agenda, the one with the highest salience fires first.**

```
rule "R4 Quiet room => suppress action"        // default salience 0
when ... then insertLogical( new SuppressedAction( $l ) ); end

rule "A1 Actuate: turn light on"
    salience -10                               // lower priority than all inference rules
when ... then modify( $light ) { setHasState( "on" ) }; end
```

- All rules share **one agenda**. Any inference activation, even one created by an actuation, fires before the next actuation.
- Java side: just `session.fireAllRules()`.
- Between rules with the **same** salience the order is **not guaranteed**. Never rely on it.
- Salience can also be an expression: `salience ( $priority * 10 )`.

Drawbacks: magic numbers spread over many files. "Inference before actuation" is only implicit in the numbers.

Mechanism 2: agenda groups

Rules are partitioned into named **agenda groups**. Only the group that has the **focus** can fire.

```
rule "R4 Quiet room => suppress action"
    agenda-group "inference"
    auto-focus true
when ... end

rule "A1 Actuate: turn light on"
    agenda-group "actuation"
when ... end
```

The focus is a **stack**; the default group `MAIN` is always at the bottom:

```
session.getAgenda().getAgendaGroup("actuation").setFocus();   // push
session.getAgenda().getAgendaGroup("inference").setFocus();   // push: runs first
session.fireAllRules();
```

```
  | inference |  <- fires until it has no activations, then is popped
  | actuation |  <- then fires
  |   MAIN    |  <- rules without an agenda-group attribute fire last
```

`auto-focus true`: when a rule gets a **new** match, its group takes the focus.
Example: A1 turns a light on, R6 (*"empty room and light on"*) now matches, and `inference` interrupts `actuation`.

Salience vs. agenda groups

	Salience	Agenda groups
Granularity	priority of individual rules	named phases (sets of rules)
Who decides the order	the numbers in the DRL files	the focus stack, controlled from Java (or `auto-focus`)
Agendas	one	one per group; only the focused one fires
Readability	implicit	explicit phases: `"inference"`, `"actuation"`
Inference re-triggered by an actuation	fires first (higher salience)	needs `auto-focus true`

A subtle difference (Exercise 2f)

`auto-focus` reacts only when a rule **gains** a match, not when a match **disappears**.
If an actuation makes an inference rule stop matching, the retraction of its `insertLogical` conclusion waits until `"inference"` has the focus again. That is why `SmartHomeApp` sets the focus and calls `fireAllRules()` **in a loop** until nothing more fires.
With salience, all rules are on one agenda, so this cannot happen.

The project

```
smart-home-drools/
├── src/main/java/ro/upb/krr/smarthome/
│   ├── SmartHomeApp.java          the house + a 6-step scenario
│   └── model/                     the ontology as Java classes (Light extends Device, ...)
└── src/main/resources/
    ├── META-INF/kmodule.xml       which rule files form each knowledge base
    └── ro/upb/krr/smarthome/rules/
        ├── salience/              01-inference.drl  02-consistency.drl  03-actuation.drl
        └── agenda/                the same rules, ordered by agenda groups
```

`kmodule.xml` selects rule files **by package (= folder)**:

```
<kbase name="salienceKB" packages="ro.upb.krr.smarthome.rules.salience">
  <ksession name="salienceSession" type="stateful" default="true"/>
</kbase>
<kbase name="agendaKB" packages="ro.upb.krr.smarthome.rules.agenda">
  <ksession name="agendaSession" type="stateful"/>
</kbase>
```

Run: `mvn compile exec:java` (salience) or `mvn compile exec:java -Dexec.args=agenda`,
or with the two VS Code launch configurations. Setup details are in `README.md`.

The scenario

Room	Initial facts
livingRoom	motion1 = on, lux1 = 12, lamp1 = off, state normal, last motion 0 min
bedroom	motion2 = on, lux2 = 5, lamp2 = off, state **quiet**
kitchen	motion3 = off, lux3 = 300, lamp3 = **on**, last motion **45** min

Step	Change
0	insert everything
1	motion1 → off
2	livingRoom lastMotionMinutes → 40
3	motion1 → on again, but lastMotionMinutes still 40 (stale data)
4	lastMotionMinutes → 0
5	bedroom state quiet → normal

After each step, `SmartHomeApp` calls `fireAllRules()` and prints the rules that fired, the derived facts and the state of the lights.

Before running, predict for each step which facts are derived and which lights change. Then run and compare.

Exercise 1: Monotonic vs. non-monotonic (slide 33)

Work in `rules/salience/` and run the salience version.

a) After step 0, which facts does Drools derive for the living room? Compare with slide 32.

b) In step 1 motion stops. Which derived facts disappear? Which rule removed them?
(Hint: none. Explain what `insertLogical` does.)

c) Lamp1 stays **on** after step 1 even though `DesiredOn(lamp1)` is gone. Why?
Which step turns it off, and through which chain of rules?

d) In `rules/salience/01-inference.drl` replace every `insertLogical` with `insert` and re-run.
Which steps now behave like SWRL? Describe one wrong behaviour this causes.
Restore the file afterwards.

Exercise 2: Conflicts and priorities (1/2)

a) Explain what happens in step 3. Which two facts are in conflict, and what does Drools do instead of declaring the knowledge base inconsistent (as an OWL reasoner would)?

b) Slide 33 asks *"DesiredOn and SuppressedAction both hold for lamp1: who wins?"*
Find the exact line in `rules/salience/03-actuation.drl` that answers this.

c) *Salience.* In `rules/salience/03-actuation.drl` remove `salience -10` from rule A1 and re-run.
Does the output change? Explain the role of salience here.
(Think about what could happen if A1 fired before R4.) Restore the file afterwards.

Exercise 2: Conflicts and priorities (2/2)

d) *Agenda groups.* Run both versions and compare the outputs:

```
mvn -q compile exec:java > salience.txt
mvn -q compile exec:java -Dexec.args=agenda > agenda.txt
diff salience.txt agenda.txt
```

Which lines differ, and does any light end up in a different state?
Explain why rules with the *same* priority may fire in a different order.

e) In `rules/agenda/01-inference.drl` remove the two lines `agenda-group "inference"` and `auto-focus true` from rule R4, so that R4 falls into the default `MAIN` group, and run the agenda version.
What happens to lamp2 in step 0? Why? (Hint: where is `MAIN` on the focus stack?)
Restore the file afterwards.

f) In `SmartHomeApp.fireWithAgendaGroups()` replace the loop with a single `fireAllRules()` call and run the agenda version. Which derived fact is left over after step 2, and why?
Restore the code afterwards.

Exercise 3: Negation as failure (slide 33, last bullet)

a) Rule R1 marks a room as occupied if *some* motion sensor is on.
Write a rule **R7** that derives `EmptyRoom(?r)` when **no motion sensor in the room is on**, something SWRL cannot state. Use `not`.

b) Write R7 using `forall` instead of `not`.
Are the two versions equivalent when a room has **no** motion sensor at all?

Add your rules to `rules/salience/01-inference.drl`.

Exercise 4: Extend the rule base

The ontology already contains `WaterLeakSensor`, `Valve` and `DesiredClose`, but no rule uses them.

a) Add a bathroom with `leak1 : WaterLeakSensor` and `valve1 : Valve` (state `"open"`) in `SmartHomeApp`.

b) Write rule **R8**: *if a water-leak sensor in a room is `"wet"`, every open valve in that room is `DesiredClose`*.

c) Write actuation rule **A5** that closes the valve and prints `valve.close_valve entity_id: valve.<id>`.

d) Should a leak be suppressed by quiet hours? Make sure your rules give the answer you chose, and justify it.

e) Add a step to the scenario in which `leak1` becomes `"wet"` and check the output.

Work in `rules/salience/`. *Optional:* add R8 and A5 to `rules/agenda/` as well.
Which attributes do they need there?